```go
package main

import (
    "flag"
    "fmt"
    "io"
    "os"
    "strconv"
    "strings"
)

// We define a "base" constant of 128, which defines the encoding range.
const base = 128

// Function to encode data using Base128.
// Takes a text string as an argument and returns a sequence of integers representing it.
// Each character from the text is converted to the corresponding number in the range 0-127.
func encodeBase128(input string) []int {
    result := make([]int, 0, len(input))
    for _, char := range input {
        result = append(result, int(char))
    }
    return result
}

// Function to decode Base128 data.
// Takes a sequence of integers as an argument and converts them to raw text.
// Each number is converted to its corresponding letter or special character.
func decodeBase128(input []int) string {
    var builder strings.Builder
    for _, code := range input {
        _, err := builder.WriteRune(rune(code))
        if err != nil {
            fmt.Fprintln(os.Stderr, "base128: error writing to builder:", err)
            os.Exit(1)
        }
    }
    return builder.String()
}

// The main function of the program that contains the logic of the program.
// Reads program call flags, such as "-e" for encoding or "-d" for decoding.
// Then it reads input from 'file' or standard input and processes it with the appropriate function.
// The result is displayed on standard output.
func main() {
    encodeFlag := flag.Bool("e", false, "Converts the input's base128 encoding into an output text
file.")
    decodeFlag := flag.Bool("d", false, "Recovers the original input file by decoding the information
that was previously encoded using base128.")
    helpFlag := flag.Bool("h", false, "Print instructions for calling and a list of available
alternatives.")
    versionFlag := flag.Bool("version", false, "Print the program's version.")
    copyrightFlag := flag.Bool("copyright", false, "Print copyright information.")
    flag.Parse()

    // Display help information and exit if the help flag is set.
    if *helpFlag {
        fmt.Println("  _________                                  ___ ________      _______  ")
        fmt.Println("  \\_____  \\_____     ______  ____  /_   |\\_____  \\  /  _  \\ ")
        fmt.Println("   |    |  _/\\__  \\   /  ___/_/ __ \\  |   | /   ____/  >     < ")
        fmt.Println("   |    |   \\/  __ \\_ \\___ \\ \\  ___/  |   |/       \\ /    _  \\ ")
        fmt.Println("   |_____  /(____  //____  > \\___  > |___|\\_______ \\/\\_____/")
        fmt.Println("     v2.0 \\/      \\/      \\/      \\/               \\/       \\/")
        fmt.Println()
        fmt.Println("base128 - base128 - encodes or decodes a text file on standard input and standard
output.")
        fmt.Println("base128 [OPTION]... [FILE input] > [FILE output]")
        fmt.Println()
        fmt.Println("EXAMPLE encode: base128 -e encode_text.txt >> decode_text.txt")
        fmt.Println("EXAMPLE decode: base128 -d decode_text.txt >> encode2_text.txt")
        fmt.Println()
        fmt.Println("Options:")
        fmt.Println()
        fmt.Println("  -d, --decode          Recovers the original input file by decoding the
information that was previously encoded using base128.")
        fmt.Println("  -e, --encode          Converts the input's base128 encoding into an output text
file.")
        fmt.Println("  -h, --help            Print instructions for calling and a list of available
alternatives.")
        fmt.Println("      --version         Print the program's version.")
        fmt.Println("      --copyright       Print copyright information.")
        fmt.Println()
        fmt.Println("(c) by Lukasz Wojcik 2023, 2025")
        fmt.Println("http://www.base128.pl")
        os.Exit(0)
    }

    // Display version information and exit if the version flag is set.
    if *versionFlag {
        fmt.Println("base128 version 1.0/2023, 2.0/2025")
        os.Exit(0)
    }

    // Display copyright information and exit if the copyright flag is set.
    if *copyrightFlag {
        fmt.Println("base128 (C) 2023, 2025 by Lukasz Wojcik")
        os.Exit(0)
    }

    // Check if both encode and decode flags are set.
    if *encodeFlag && *decodeFlag {
        fmt.Fprintln(os.Stderr, "base128: cannot specify both -e and -d flags. Please use -e for ENCODE
or -d for DECODE or -h for HELP")
        os.Exit(1)
    }

    // Check if neither encode nor decode flags are set.
    if !*encodeFlag && !*decodeFlag {
        fmt.Fprintln(os.Stderr, "base128: no action specified. Please use -e for ENCODE or -d for
DECODE or -h for HELP")
        os.Exit(1)
    }

    // Read input from file or standard input.
    args := flag.Args()
    var input string
    if len(args) == 0 || args[0] == "-" {
        bytes, err := io.ReadAll(os.Stdin)
        if err != nil {
            fmt.Fprintln(os.Stderr, "base128: error reading from stdin:", err)
            os.Exit(1)
        }
        input = string(bytes)
    } else {
        bytes, err := os.ReadFile(args[0])
        if err != nil {
            fmt.Fprintln(os.Stderr, "base128: error reading file:", err)
            os.Exit(1)
        }
        input = string(bytes)
    }

    // Process input based on the selected flag.
    if *encodeFlag {
        numbers := encodeBase128(input)
        output := strings.Trim(strings.Join(strings.Fields(fmt.Sprint(numbers)), " "), "[]")
        fmt.Println(output)
    } else if *decodeFlag {
        numbersStr := strings.Fields(input)
        numbers := make([]int, len(numbersStr))
        for i, numberStr := range numbersStr {
            number, err := strconv.Atoi(numberStr)
            if err != nil {
                fmt.Fprintln(os.Stderr, "base128: error decoding number:", err)
                os.Exit(1)
            }
            numbers[i] = number
        }
        output := decodeBase128(numbers)
        fmt.Println(output)
    }
}
```